



Getting Started

# 101 on Portkey's Gateway Configs

 Copy page



You are likely familiar with how to make an API call to GPT4 for chat completions.

However, did you know you can **set up** automatic retries for requests that might fail on OpenAI's end using Portkey?

The Portkey AI gateway provides several useful features that you can use to enhance your requests. In this cookbook, we will start by making an API call to LLM and explore how Gateway Configs can be utilized to optimize these API calls.

## 1. API calls to LLMs with Portkey

Consider a typical API call to GPT4 to get chat completions using OpenAI SDK. It takes `messages` and `model` arguments to get us a response. If you have tried one before, the following code snippet should look familiar. That's because Portkey Client SDK follows the same signature as OpenAI's.



```
const response = await portkey.chat.completions.create({
  model: '@openai-prod/gpt-5-mini', // @provider-slug/model-name
  messages: [{role: 'user', content: 'What are the 7 wonders of the world?'}]
});

console.log(response.choices[0].message.content);
```

Along with your Portkey API Key ([get one](#)), you'll need to add a provider in [Model Catalog](#). Portkey securely stores your LLM provider API keys and lets you reference them using provider slugs like `@openai-prod`.

With basics out of our way, let's jump into applying what we set out to do in the first place with the AI gateway — To automatically retry our request when we hit rate-limits (429 status codes).

## 2. Apply Gateway Configs

The AI gateway requires instructions to automatically retry requests. This involves providing Gateway Configs, which are essentially JSON objects that orchestrate the AI gateway. In our current scenario, we are targeting GPT4 with requests that have automatic retries on 429 status codes.

Config

```
{
  "retry": {"attempts": 3, "on_status_codes": [429]}
}
```

We now have our Gateway Configs sorted. But how do we instruct our AI gateway?



automatically apply in the request headers to instruct the AI gateway. UI builder features lint suggestions, makes it easy to reference (through config ID), eliminates manual management, and allows you to view version history.

To create Gateway Configs,

1. Go to **portkey.ai** and
2. Click on **Configs**
  - a. Select **Create**
  - b. Choose any name (such as request\_retries)

Write the configs in the playground and click **Save Config**:

See the saved configs in the list along with the ID :

Try it out now!

The Configs saved will appear as a row item on the Configs page. The ID is important as it is referenced in our calls through the AI gateway.

## Portkey SDK

The Portkey SDK accepts the config parameter that takes the created config ID as it's argument. To ensure all requests have automatic retries enabled on them, pass the config ID as argument when `portkey` is instantiated.

That's right! One line of code, and all the request from your apps now inherit Gateway Configs and demonstrate automatic retries.

Let's take a look at the code snippet:



```
});  
  
const response = await portkey.chat.completions.create({  
  model: '@openai-prod/gpt-5-mini',  
  messages: [{role: 'user', content: 'What are the 7 wonders of the world?'}]  
});  
  
console.log(response.choices[0].message.content);
```

## Axios

In the cases, where you are not able to use an SDK, you can pass the same configs as headers with the key `x-portkey-config`.

 Axios

```
const CONFIG_ID = 'pc-reques-edf21c';  
const PORTKEY_API_KEY = 'xxxxxrk';  
const OPENAI_API_KEY = 'sk-*****';  
  
const response = await axios({  
  method: 'post',  
  url: 'https://api.portkey.ai/v1/chat/completions',  
  headers: {  
    'Content-Type': 'application/json',  
    Authorization: `Bearer ${OPENAI_API_KEY}`,  
    'x-portkey-api-key': PORTKEY_API_KEY,  
    'x-portkey-provider': 'openai',  
    'x-portkey-config': CONFIG_ID  
  },  
  data: data  
});  
  
console.log(response.data);
```



OpenAI JS

```
import OpenAI from 'openai';
import { PORTKEY_GATEWAY_URL, createHeaders } from 'portkey-ai';

const PORTKEY_API_KEY = 'xxxxxrk';
const CONFIG_ID = 'pc-reques-edf21c';

const messages = [{role: 'user', content: 'What are the 7 wonders of the world'}];

const openai = new OpenAI({
  apiKey: PORTKEY_API_KEY,
  baseURL: PORTKEY_GATEWAY_URL,
  defaultHeaders: createHeaders({
    config: CONFIG_ID
  })
});

const chatCompletion = await openai.chat.completions.create({
  model: '@openai-prod/gpt-5-mini',
  messages
});

console.log(chatCompletion.choices[0].message.content);
```

The approach to declare the Gateway Configs in the UI and reference them in the code is recommended since it keeps the Configs atomic and decoupled from the business logic and can be upgraded to add more features. What if you want to enable caching for all your thousands of requests? Just update the Configs from the UI. No commits. No redeployes.

## b. Reference Gateway Configs in the Code



Introducing Agent Gateway — governance, observability, and control for your AI agents.  
[Register for live webinar ↗](#)



JS JavaScript

```
import { Portkey } from 'portkey-ai';

const portkey = new Portkey({
  apiKey: 'PORTKEY_API_KEY',
  config: JSON.stringify({
    retry: {attempts: 3, on_status_codes: [429]}
  })
});

const response = await portkey.chat.completions.create({
  model: '@openai-prod/gpt-5-mini',
  messages: [{role: 'user', content: 'What are the 7 wonders of the world?'}]
});

console.log(response.choices[0].message.content);
```

## Axios



```
const PORTKEY_API_KEY = 'xxxxxxxx';
const OPENAI_API_KEY = 'sk-xxxxxxxx';

const data = {
  model: 'gpt-5-mini',
  messages: [{role: 'user', content: 'What are 7 wonders of the world?'}]
};

const { data: response } = await axios({
  method: 'post',
  url: 'https://api.portkey.ai/v1/chat/completions',
  headers: {
    'Content-Type': 'application/json',
    Authorization: `Bearer ${OPENAI_API_KEY}`,
    'x-portkey-api-key': PORTKEY_API_KEY,
    'x-portkey-provider': 'openai',
    'x-portkey-config': JSON.stringify(CONFIG_ID)
  },
  data: data
});

console.log(response.choices[0].message.content);
```

## OpenAI SDK



```
const messages = [{role: 'user', content: 'What are the / wonders of the world'}]

const openai = new OpenAI({
  apiKey: PORTKEY_API_KEY,
  baseUrl: PORTKEY_GATEWAY_URL,
  defaultHeaders: createHeaders({
    config: {
      retry: {attempts: 3, on_status_codes: [429]}
    }
  })
});

const chatCompletion = await openai.chat.completions.create({
  model: '@openai-prod/gpt-5-mini',
  messages
});

console.log(chatCompletion.choices[0].message.content);
```

Those are three ways to use Gateway Configs in your requests.

In the cases where you want to specifically add a config for a specific request instead of all, Portkey allows you to pass `config` argument as separate objects right at the time of chat completions call instead of `Portkey({..})` instantiation.

 JavaScript

```
const response = await portkey.chat.completions.create({
  messages,
  model: 'gpt-5-mini'
}, {
  config: 'config_id' // or expanded Config Object
});
```





Gateway Configs is a tool that can help you manage fallbacks, request timeouts, load balancing, caching, and more. With Portkey's support for 1,600+ LLMs, it is a powerful tool for managing complex use cases that involve multiple target configurations. A Gateway Config that encompasses such complexity may look like:

```
TARGET 1 (root):  
  OpenAI GPT4  
  Simple Cache  
  On 429:  
    TARGET 2 (loadbalance):  
      Anthropic Claude3  
      Semantic Cache  
      On 5XX  
    TARGET 3 (loadbalance):  
      Anyscale Mixtral 7B  
      On 4XX, 5XX  
      TARGET 4 (fallback):  
        Llama Models  
        Automatic Retries  
        Request Timeouts
```

For complete reference, refer to the [Config Object](#).

It's exciting to see all the AI gateway features available for your requests. Feel free to experiment and make the most of them. Keep up the great work!

Last modified on February 4, 2026